Forgettable Athlete — Simple Explainer

A noob-friendly guide to the injury prediction project: model ranking, ROC-AUC vs accuracy, correlation heatmap, SHAP, and machine unlearning (SISA).

1. Which model won, and the ranking

Rank	Model	Accuracy	F1	ROC-AUC
1	Logistic Regression	0.9500	0.9500	0.9961
2	Random Forest	0.9563	0.9560	0.9925
3	LightGBM	0.9187	0.9161	0.9881
4	XGBoost	0.9375	0.9359	0.9848

Logistic Regression took first place on ROC-AUC. Random Forest is right behind it and even beats it on raw accuracy. LightGBM came third and XGBoost was fourth.

2. Why ROC-AUC instead of Accuracy?

Accuracy only asks: did the model pick the right label at a 50% threshold? It treats a 51% confident guess and a 99% confident guess the same.

ROC-AUC asks something deeper: across *every* possible cutoff, how well does the model separate injured from healthy players? A score of 1.0 means perfect ranking; 0.5 means random.

Why this matters here:

- In sports medicine, the cost of missing an injury (false negative) is different from over-resting a player (false positive). A coach may want to use a stricter or looser threshold than 50%.
- ROC-AUC tells us how good the model is at *ranking* risk — useful no matter where the coach sets the threshold.
- It is also threshold-independent and works well even if classes are imbalanced.

Think of accuracy as 'did you pass the test?' and ROC-AUC as 'how confidently were your answers right?'

3. Why Logistic Regression won (the simple version)

Before training, we created smart features by hand: **ACWR** (workload ratio), **Recovery_Score**, **Fatigue_Index**, and **Injury_Frequency_Score**. These already mix sleep, stress, workload, and prior injuries into meaningful numbers.

Other reasons Logistic Regression came out on top:

- **Small dataset (800 players).** Simple models beat complex ones when data is limited — fewer parameters means less overfitting.
- **Calibrated probabilities.** Logistic Regression outputs natural probabilities (0 to 1), which is exactly what ROC-AUC rewards.
- **Pre-cooked features.** The non-linear patterns were already baked into the engineered columns, so a straight-line decision was enough.
- **Stable training.** Linear models are not picky about hyperparameters; trees and boosting need careful tuning we didn't do.
- **Less noise sensitivity.** With only 640 training rows, boosting models started fitting noise — logistic regression ignored it.

4. Correlation Analysis — reading the heatmap

A correlation heatmap is a grid of colored squares. Each square shows how strongly two features move together. The numbers go from -1 to +1.

How to read the colors:

- **Deep red (+1):** when one feature goes up, the other goes up too.
- **Deep blue (-1):** when one goes up, the other goes down.
- **Near white (0):** no relationship.
- The diagonal is always +1 because every feature perfectly correlates with itself.

What we learned from our heatmap:

- **No single feature is a giveaway.** The biggest correlations with Injury_Next_Season are modest (around 0.1 to 0.3). That tells us injury is a combination story, not a one-cause story.
- **Our engineered features lead.** Fatigue_Index and Injury_Frequency_Score top the list, proving the feature engineering added real signal.
- **Physical tests cluster together.** Knee strength, hamstring flexibility, balance, agility — fit athletes score well across the board, so these features overlap. Models will pick one and ignore the redundant ones.

Quick rule of thumb: |corr| > 0.7 = strong, 0.3 to 0.7 = moderate, < 0.3 = weak.

5. SHAP / Shapley values — explained simply

Imagine the model is a team and each feature is a teammate. After the team scores (makes a prediction), Shapley values fairly split the credit: how much did each teammate contribute? This idea comes from game theory and Nobel laureate Lloyd Shapley.

The math in one line: Final prediction = average risk + sum of every feature's SHAP value.

The summary (beeswarm) plot:

- Y-axis: features ranked by importance (top = most important).
- X-axis: SHAP value — right of zero = pushes toward 'injured', left = pushes toward 'healthy'.
- Color: red dot = high feature value, blue dot = low feature value.
- Each dot = one player.

Example: in the Previous_Injury_Count row, red dots cluster on the right — meaning a high prior-injury count pushes the model to predict injury. In Recovery_Score, red dots cluster on the left — high recovery protects the player.

The bar plot: the same info, but only shows the average size of impact per feature — pure importance ranking.

The waterfall plot: for one single player, it stacks each feature's contribution from the base rate up to the final risk number. Red bars push risk up, blue bars push it down. This is what a medic would show a player to justify a recommendation.

Why this matters: it turns the model from a black box into an auditable decision-maker. Useful for GDPR, useful for coaches.

6. Machine Unlearning (SISA) — explained simply

Under GDPR, a player can say: *'Delete my data and remove my influence from your model.'* Retraining the whole model from scratch every time someone asks is slow and expensive. SISA solves this.

SISA = Sharded, Isolated, Sliced, Aggregated.

- **Sharded:** Split the training data into N small chunks (we used 5 chunks).
- **Isolated:** Train one separate mini-model on each chunk. No chunk shares data with another.
- **Sliced:** (optional) save checkpoints inside each chunk so you can rewind quickly.
- **Aggregated:** When predicting, average the answers from all mini-models.

The unlearning trick: If player X wants to be forgotten, only the chunk they belonged to needs retraining. Other chunks stay untouched. Expected speedup: roughly N times faster.

Method	Accuracy	ROC-AUC	Time (s)
Full retraining	0.9563	0.9922	0.148
SISA unlearning	0.9375	0.9855	0.116

Result: SISA was about 22% faster while losing almost no accuracy. The speedup was modest because 54 deletion requests were spread across all 5 chunks — every chunk needed retraining. If deletion requests were grouped in fewer chunks, the speedup would be much bigger.

The trade-off: Each mini-model sees less data, so the ensemble is slightly weaker than one giant model — but in return we get fast, provable forgetting.

7. TL;DR

- Logistic Regression won (ROC-AUC = 0.9961). Random Forest 2nd, LightGBM 3rd, XGBoost 4th.
- ROC-AUC was preferred over accuracy because it measures ranking quality across all thresholds.
- The smart engineered features (Fatigue, Recovery, ACWR, Injury Frequency) gave Logistic Regression an edge.
- The heatmap showed no single feature dominates — injury is a multi-factor story.
- SHAP makes individual predictions explainable using a game-theory credit-splitting idea.
- SISA lets us delete a player's influence quickly without retraining everything from scratch.